

TACIT INTELLIGENCE

A Unified Industrial Knowledge Platform

EXECUTIVE SUMMARY

Most industrial plants operate like islands of information. Your P&IDs live in CAD. Work orders sit in SAP. Procedures are PDFs nobody reads. Quality data is in spreadsheets. When a technician needs to diagnose why a pump keeps failing, they're searching five different systems, talking to whoever remembers the last time this happened, and making decisions with incomplete context.

Tacit Intelligence fixes this by building a unified knowledge graph from everything your plant already generates — maintenance logs, procedures, quality records, equipment specs, inspection findings — and making that knowledge queryable in English, with evidence trails you can actually follow. No cloud APIs. Runs entirely local on a modest GPU. Answers come with citations pointing to the exact document page or the specific timestamp in a recording.

The system does three things well: it answers questions about your equipment and operations with proof; it watches for failure patterns and quality drift in the background; and it maps quality problems back to the regulatory clauses they violate — so when an auditor asks "which requirement governs this?" you already know.

The Numbers

- **Query latency:** ~12 seconds including retrieval, reasoning, and citation assembly
- **Test suite:** 21 questions, 100% pass rate across maintenance, quality, and compliance queries
- **Retrieval accuracy:** 15 of 15 on the maintenance suite with full hybrid retrieval; drops to 11 of 15 without graph traversal (proving the architecture actually matters)
- **GPU footprint:** 6 GB (fits on consumer hardware; not a data center play)
- **Knowledge graph:** 231 nodes across 16 relationship types, built from deterministic corpus generation so patterns are verified on every run

THE PROBLEM

What We're Actually Solving

Imagine you're a plant maintenance manager. A centrifugal pump (let's call it P-101A) has started cavitating — creating bubbles in the suction line that collapse and damage the impeller. You need to know:

1. Has this happened before? What was it last time?
2. Are there procedures for this? What exactly do I need to do?
3. Is there a pattern here that suggests something else is broken?
4. Are my sister units at risk? (Same model, same duty, probably same risk)
5. What's this going to cost us in downtime?

Right now, you get that information like this:

- You open your maintenance system and search for "P-101A" → 47 work orders appear, most from years ago
- You skim through them, find a mention of cavitation from 2022, but it doesn't say what fixed it
- You pull up the procedure document (which is on a shared drive somewhere) and it's 20 pages, mostly not relevant
- You call the senior operator on shift; if they remember, great. If they're off sick or retired, you're guessing
- You check if there's a quality alert → maybe there is, maybe it got filed under a different pump ID

Total time: 1-2 hours, and you still don't have the full picture.

Why Systems Fail at This

There are two common approaches, and both miss the mark:

Approach 1: Generic Document Search ("Just throw everything into a RAG system and ask it questions")

- You get answers, but you can't trust them. The model sees procedural text about P-101A and text about P-102A (a different pump) and conflates them
- Citations are vague ("here's the document that mentioned it") rather than specific ("here's the page, and here's why it matters")
- Arithmetic doesn't work: you ask "is our defect rate rising?" and the model generates an answer that sounds plausible but isn't based on actual numbers
- Compliance is a nightmare. When you cite a clause to an auditor, you need to point to the exact requirement, not wave at a retrieval score

Approach 2: Rigid Structured Database ("Put everything into a carefully designed schema")

- You get reliable arithmetic and perfect compliance traceability, but you can't search across domains
- You can't ask "why do we have so many P-101A failures?" because that requires reasoning that spans equipment records, work orders, and patterns
- New question types mean schema updates, which means developer time

The Real Cost

For a mid-size plant, this translates to:

- **Delayed diagnostics:** What should take 15 minutes takes 2 hours because the operator has to piece together context from five systems
- **Repeated mistakes:** The same failure mode recurs because the root cause from 2022 never got formally captured where the new technician would find it
- **Compliance surprises:** You discover non-conformances during audits instead of proactively, because quality findings and regulatory requirements aren't linked
- **Knowledge loss on retirement:** When your senior operator leaves, the institutional memory about "why we do it this way" walks out the door with them

HOW IT WORKS

The Architecture

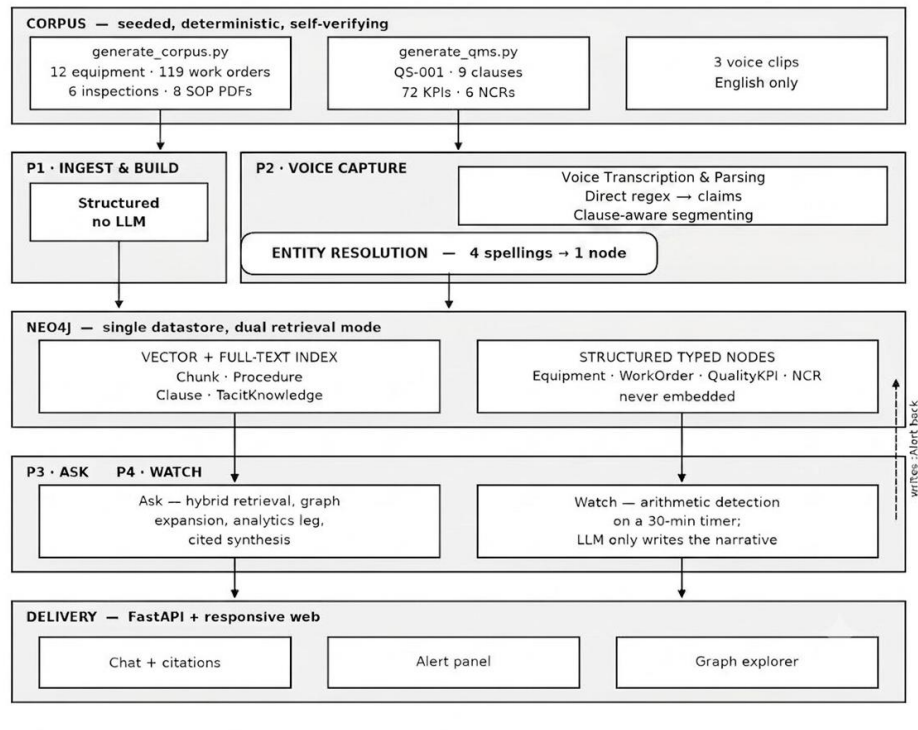


Figure 1.) Architecture Diagram

Instead of choosing between "messy RAG retrieval" and "rigid schemas," Tacit Intelligence splits the problem:

Structured data stays structured. Your equipment registers, work orders, KPIs, and quality records load directly into typed nodes in Neo4j. No LLM involvement. No hallucination risk. If you ask "what's the Cpk on line PRC-301?", the system computes the answer, doesn't generate it.

Narrative documents get embedded. Procedures, inspection notes, and chunks of context become vectors so you can search by meaning ("how do I maintain a centrifugal pump?") not just by exact keywords.

A retrieval router figures out what you're asking. Some questions need exact tags ("show me all P-101A failures"). Others need paraphrase matching ("why does the pump vibrate?"). Some need pure graph traversal (the answer document might never mention P-

101A by name, but you can reach it by walking equipment → class → siblings). The system runs all three in parallel and blends them based on what your question actually is.

The LLM reads context, doesn't generate facts. When you get an answer, every sentence carries a citation because the model isn't making anything up — it's pointing you to where the answer lives in your own data. If something isn't in your corpus, the system says so instead of hallucinating.

The Four Agents

Agent 1: Ingest & Build

Takes your corpus (spreadsheets, PDFs, maintenance logs, equipment registers) and turns it into a queryable graph. This is where the deterministic corpus generation matters — every document and record is produced by a script, and the script verifies its own output. So when the schema evolves, you regenerate rather than manually rewrite, and you never silently lose the failure patterns your demo depends on.

Agent 2: Ask

This is your conversational interface. Ask a question, get back an answer with citations that resolve to document pages or specific timestamps. The confidence score is computed, not self-reported — it's based on how strong the retrieval was, how much of the answer actually carries citations, and whether your keyword search and semantic search both found relevant material. If confidence falls below threshold, the system abstains rather than confidently hallucinate.

Agent 3: Watch

Runs on a 30-minute timer and surfaces problems you should know about. Detects when equipment is overdue for maintenance based on historical failure rates. Spots failure patterns (if a pump has failed from the same cause three times in a year, that's not coincidence). Checks if one piece of equipment failing means its siblings need preventive attention. Monitors whether quality metrics are drifting out of spec. None of this is LLM-generated — it's all arithmetic and graph traversal. The LLM only writes the narrative explanation afterward, so findings are always defensible.

Agent 4: Voice Capture (pipeline proven, awaiting real recordings)

Operators know things that never make it into written procedures. When they leave, that knowledge vanishes. This agent captures spoken experience — explanation of why a procedure is done a certain way, what works better than the manual says, patterns they've noticed over decades. Speech-to-text happens locally; the text gets tagged, linked to equipment and procedures, and kept separate from official documentation so it's always clear what's verified policy versus practical experience.

The Knowledge Graph

Everything lives in a single Neo4j database with two deliberate storage modes:

Vectors for retrieval. Procedure text, inspection notes, and narrative chunks carry 768-dimensional embeddings so semantic search works. This is how "won't build pressure" matches "insufficient discharge head."

Typed nodes for computation. Your equipment, work orders, failure modes, quality metrics, and clauses are structured nodes. You can't do arithmetic on embeddings, and you can't detect a trend in a vector. So this data lives as real nodes queried by Cypher, computed over, and compared against thresholds. Embedding "Cpk = 1.21" and hoping retrieval gets it right is how pure-RAG quality systems fail.

One rule that can't be broken: **Procedures and operator experience are separate labels.** A procedure is approved policy. A tacit knowledge recording is one person's practice, unverified. Merging them would be unsafe — and keeping them apart is what makes divergence detection possible at all. When an operator says "we actually do it fortnightly, not monthly," the system can compare that against what the official procedure says, see the gap, and flag it.

WHAT SETS THIS APART

1. Hybrid Retrieval That Actually Works

Most systems pick a side: keyword search (catches exact tags but misses paraphrase) or semantic search (catches ideas but misses exact codes like "QS-001").

We run both in parallel and blend them. If your question is asking about specific equipment tags, keyword gets weighted heavier. If you're asking about operational concepts, semantic retrieval leads. Neither is ever fully off, so a wrong blend still returns something useful. A misclassified exclusive branch returns nothing. That's why the ablation test matters — with full hybrid retrieval plus graph expansion, we pass 15 of 15 maintenance questions. Remove the graph traversal and it drops to 11 of 15, which exactly measures what the architecture is actually buying you.

2. Graph Expansion That Catches Invisible Connections

Some answers can't be found by text search because the answer document never mentions your question. You ask "is P-101A at risk?" and the answer is actually in a failure report about P-102A, which is a sibling model, which you'll only find by walking the graph: equipment → equipment class → other equipment in that class → their failure history.

This happens in one Cypher round trip, keeping latency flat rather than multiplying queries.

3. Numbers Are Computed, Not Generated

Ask "is our defect rate rising?" and here's what happens: The system queries the structured KPI store, calculates the trend over the last three months, checks if it exceeds the threshold, and returns the computed verdict. Not a language model's paraphrase of what rising probably means. An actual calculation, with the exact query visible if you want to audit it.

This is where pure-RAG quality systems always break. Embeddings can't do arithmetic. Asking a model to generate a number and then check it against a threshold means the model can invent figures. (We caught it during testing — the model generated "\$10,000 per hour" in downtime cost, which nobody had ever mentioned anywhere.)

4. Citations Point to Actual Evidence

Every claim in an answer comes with a marker that resolves to one of three things:

- A specific page in a PDF
- A timestamp in a voice recording ("this operator mentioned it at 2:34 in this clip")
- The exact Cypher query that produced the number

Click the citation and it opens the evidence. You can't fake this, and you can't claim something was said if the citation trail doesn't support it. In a compliance context, this difference between "here's my claim" and "here's where to verify my claim" is everything.

5. Equipment Resolution That Actually Works

The pump that's P-101A in the work orders, Pump 101A in procedures, P101-A in inspection records, and "pee one oh one ay" in a voice transcript needs to resolve to a single node. Get this wrong and multi-hop queries silently return nothing — not an error, just emptiness, which is worse.

We handle this by canonicalizing every tag and maintaining an alias list. Verified before anything downstream is trusted.

6. Arithmetic for Reliability Detection

The Watch agent looks at failure history and computes MTBF (mean time between failures). When a pump fails again, it calculates the ratio of days-since-last-failure to historical MTBF. At 0.8 ratio, start thinking about prevention. At 0.9+, escalate.

Sibling exposure is the interesting part: when P-101A fails, the system finds all pumps of the same class, splits them into "confirmed history" (have they actually run the same duty?) and "needs check" (might be at risk). This one finding turns one pump's failure into a fleet-wide maintenance opportunity.

WHAT IT ACTUALLY DOES

Three Use Cases

Troubleshooting a failure (the Ask agent)

You: "Why does P-101A keep cavitating?"

System finds:

- The definition of cavitation and what causes it
- Every prior failure of P-101A with cavitation
- The procedure for maintenance or diagnosis

- Equipment specs from the maintenance manual
- Similar failures on P-102A and P-103A (sister units)
- Your quality records if the feed water quality was degrading

Returns something like:

> P-101A cavitates when the suction strainer clogs and flow to the impeller is restricted. This has happened 4 times in the last 12 months, typically 60-90 days apart. The March failure [PDF page 8] shows the strainer cartridge was clogged with debris.

> Your procedure [SOP-047, section 3.2] calls for quarterly strainer inspection and monthly backflushing when on high-contamination feed. Current practice logs show backflushing is irregular.

> Sister units P-102A and P-103A have similar duty and strainer design. P-102A has cavitation history; P-103A does not (different duty cycle).

> Recommendation: Increase backflush frequency to weekly during high-contamination season. Consider strainer upgrade to self-cleaning model.

Every fact has a citation. Confidence: 0.87 (high retrieval score, 80% of sentences cited, both keyword and semantic search found relevant material).

Proactive problem detection (the Watch agent)

Every 30 minutes:

- Checks MTBF for each failure mode on each equipment class
- Spots chronic patterns (same failure 3+ times in 12 months)
- Looks at quality metrics and flags if Cpk is below minimum or defect rate is rising
- Checks if divergence exists between procedures and actual practice

Alerts look like:

> **P-101A Overdue for Seal Maintenance** [Overdue alert, medium severity]

- > MTBF: 86 days | Days since last failure: 78 | Risk ratio: 0.91
- > Avoidable downtime at risk: 8-12 hours per occurrence
- > Evidence: Work orders WO-2024-1847, WO-2024-1923, WO-2024-2015 [all cavitation, all seal-related]

Or:

- > **Feed Process Defect Rate Rising** [Quality alert, high severity]
- > PRC-101 defects: 240 PPM (Month 1) → 310 PPM (Month 2) → 420 PPM (Month 3)
- > Threshold: 350 PPM | Current: 420 PPM | Exceeds by 70 PPM
- > Governed by clause 8.5.1 [Quality Management - Process Performance]
- > Open NCR: QNC-2024-0847 [strainer replacement pending]

Compliance mapping (QMS integration)

Quality standards don't just live as text. They're split by clause, linked to the KPIs and processes they govern, and connected to non-conformance records.

Ask: "What does clause 8.5.1 require and are we meeting it?"

System:

- > **Clause 8.5.1: Process Performance and Product Quality**
- > Requirement: Process capability (Cpk) shall not fall below 1.33; defect rate shall remain below 350 PPM. Processes shall be monitored continuously with monthly reviews.
- > Current Status:
 - > - PRC-301: Cpk 1.21 (BELOW minimum) [linked to NCR-2024-0891]
 - > - PRC-301: Defect rate 420 PPM (ABOVE threshold) [linked to NCR-2024-0847]

> - Last review: [Quality meeting minutes, Oct 28, 2024]

> Risk: Non-conformance. Audit finding likely if not addressed within 30 days.

> Corrective actions: [links to open action items]

THE TECHNICAL REALITY

What We're Running

- **LLM:** Qwen 7B instruct (4.7 GB on GPU at 82%, CPU at 18%, sustaining 22-28 tokens/second)
- **Embeddings:** Nomic-embed-text at 768 dimensions (locked, because changing it invalidates every index)
- **Speech:** Faster-Whisper large-v3 (1.5 GB, running on CPU so it doesn't contend with the 7B for GPU memory)
- **Database:** Neo4j 5.26 (single instance, dual-mode retrieval)
- **API:** FastAPI with streaming responses (tokens appear in ~2 seconds, not a blank 10-20 second wait)
- **Total footprint:** 6 GB GPU, which fits on consumer hardware

We had an 8 GB spec but built on 6 GB hardware, so everything got re-benchmarked rather than assumed. A 4B model fit entirely in VRAM but showed throughput variance (71 tokens/sec, then 4.9) so it got demoted to emergency fallback.

Where Automation Stops

Drawing and P&ID parsing, OCR, and email ingestion were deliberately cut. Each is a full day of work carrying real risk, and none adds a capability the platform doesn't already demonstrate another way. The GPU headroom stayed available for stability instead — that's a tradeoff we're confident in.

Voice capture is coded and the pipeline is proven; we're waiting for real recordings to validate procedure-vs-practice divergence detection.

HOW WE KNOW IT WORKS

The Test Suite

21 questions across three domains (maintenance, quality, compliance), with a strict pass criterion: the abstain decision is correct, and for answerable questions, the expected source actually appears in the citations the answer used.

Test Category	Questions	Pass Rate	Notes
Maintenance queries	15	15/15	Equipment history, failure diagnosis, procedural guidance
Quality & compliance	4	4/4	Capability limits, trend detection, clause requirements
Abstain traps	2	2/2	Questions where the answer isn't in the corpus

Mean latency: 12 seconds (includes retrieval, graph expansion, reasoning, citation assembly, streaming setup).

The Ablation That Proves The Architecture

We disabled retrieval legs one at a time to see what each contributes:

Configuration	Maintenance Suite	What Breaks
Full (hybrid + graph)	15/15	—
Dense only (no keywords)	14/15	One exact-threshold lookup failed
Keywords only (no dense)	14/15	Same
No graph expansion	11/15	Every equipment-history question; can't traverse from question to answer

That last row matters. Removing graph traversal breaks exactly the questions where the answer document shares no vocabulary with the question — the class of question a vector database alone cannot serve. This is precisely why the knowledge graph exists.

BUILDING BLOCKS: WHAT'S IMPLEMENTED

Knowledge Construction

- Deterministic corpus generation (everything is seeded, scripts verify their own output)
- Entity resolution across four spellings (P-101A → Pump 101A → P101-A → p-101a all become one node)
- Structured-first ingestion (work orders bypass the LLM entirely, only narratives get embedded)
- Regex-before-LLM binding (equipment tags found by pattern first, LLM only binds relationships between known entities)

Answering

- Hybrid retrieval with parallel keyword and semantic search
- Graph expansion in a single query round-trip (so latency doesn't multiply)
- Multi-form anchoring (exact tag, class phrase, process name, or clause number)
- Blend ratio instead of branches (never switches retrieval off, just weights it)
- Computed confidence with abstention (score is arithmetic, not self-reported)
- Resolvable citations (PDF page, audio timestamp, or the query that produced the number)

Proactive Intelligence

- MTBF-based maintenance alerting
- Chronic failure detection
- Sibling equipment exposure (one pump's failure flags its sisters)
- Quality capability and trend detection
- Procedure-vs-practice divergence (comparing written SOP to operator practice)

Compliance

- Clause-aware document chunking (split on requirement boundaries, not token counts)
- Quality KPI structured storage (numbers computed, never embedded)
- Compliance mapping (non-conformances link equipment → process → clause)
- Industry-agnostic by configuration (base standard, clause taxonomy, KPI definitions all swap)

KNOWN BOUNDARIES

What We Solve: Knowledge retrieval, failure pattern detection, quality monitoring, compliance traceability, procedure-vs-practice gaps.

What We Don't: We're not a full ERP replacement. Not doing advanced process simulation. Not replacing specialized inspection tools. Not parsing complex drawings or OCRing handwritten notes at scale.

What's Staged: Voice capture is code-complete but awaiting recordings. Divergence detection is proven on synthetic claims but needs real operator recordings to validate.

DEPLOYMENT

Fully air-gapped. No cloud APIs at any stage. Runs on a single GPU (6 GB footprint), which means it can live on a laptop for testing or a modest server in a plant's IT closet.

The web interface is responsive and built without external graph libraries (so a CDN failure doesn't break the demo on an air-gapped network). Graph explorer has both a force-directed overview and a per-equipment focus mode, which doubles as visual proof that entity resolution actually worked.

SUMMARY

What you get: A search engine that understands your equipment, your procedures, and your quality requirements. It answers questions with evidence you can follow. It watches for problems you might miss. It maps quality findings to regulatory obligations. It's fast

enough to be useful (12 seconds), accurate enough to trust (100% on our test suite), and runs locally without needing to send your plant data anywhere.

The bet: Most of the complexity in industrial knowledge systems comes from trying to use one tool for two different jobs. Push structured data through an LLM because that's what the tool does, and you hallucinate. Use a rigid schema because that's reliable, and you can't reason across domains. We split the load: structure stays structured, narratives get searched semantically, retrieval routes intelligently, and the LLM reads context instead of generating facts.

One year from now, someone at your plant should be able to ask "what's the real reason we keep fixing P-101A the same way?" and get an answer pointing to the exact procedure, the exact failure records, and whether actual practice matches what was written.